

Reviewer Pack — Minimal p-Adic Valuation of Affine Plane Curve Points vs AC0...

Entry d6756f053a72 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 00:35:46 UTC

Minimal p-Adic Valuation of Affine Plane Curve Points vs AC0 Circuit Lower Bounds

Entry ID: d6756f053a72 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 00:35:46 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): AC^0 Circuit Complexity

Statement:

[‘For every affine plane curve C over a finite field with points $P_1, \dots, P_k$ on C , the minimal p-adic valuation of the coordinates of any point P_i is lower-bounded by the number of AND gates in any AC^0 circuit computing the characteristic function of C .’, ‘Equivalently, for any affine plane curve C and any AC^0 circuit c with n inputs that computes its characteristic function, there exists a point P on C such that the p-adic valuation of each coordinate of P is at least $\log_2(n)$.’]

Rationale (proposer’s reasoning):

[‘p-adic analysis provides a way to measure the complexity of geometric objects in terms of arithmetic operations, which could reveal underlying structures related to computational complexity.’, ‘The conjecture suggests that the arithmetic structure of an affine plane curve is reflected in its representation by AC^0 circuits, potentially leading to insights into circuit lower bounds.’]

Taxonomy category: p-adic_analysis (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f3b8df5c9eb35c79

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all affine plane curves C and their characteristic functions computed by AC^0 circuits, the minimal p-adic valuation of curve points meets or exceeds $\log_2(n)$ where n is the number of AND gates in the circuit for at least 80% of the seeds. The conjecture is falsified if for any seed, there exists a point P_i on C with a p-adic valuation below $\log_2(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic valuation affine plane curve AND AC^0 circuit complexity - characteristic function AC^0 lower bound AND p-adic analysis - AND gate count related to p-adic valuation of points on curves

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/1010.0349v2>] On p-adic lattices and Grassmannians - [<http://arxiv.org/abs/2408.00810v3>] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [<http://arxiv.org/abs/1501.05760v2>] On integrality of p -adic iterated integrals - [<http://arxiv.org/abs/1503.08756v4>] A bound on the norm of overconvergent p -adic multiple polylogarithms

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random finite field size p (prime number)
    p = next_prime()

    # Generate a random affine plane curve over  $F_p$ 
    n = 5 + random.randint(0, 24) # n in {5, 10, ..., 30}
    points = generate_points(n, p)

    # Compute the characteristic function for the curve
    circuit_size = len(generate_circuit(points))

    # Calculate the minimal p-adic valuation of the coordinates of any point
    min_valuation = min(p_adic_valuation(point, p) for point in points)

    # Check if the conjecture holds
    conjecture_holds = min_valuation >= math.log2(circuit_size)
    counterexample = "" if conjecture_holds else f"Point with valuation {min_valuation} < log2({circuit_size})"
```

```

    return {
        "metric_name": "Minimal p-Adic Valuation",
        "metric_value": min_valuation,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def next_prime():
    def is_prime(num):
        if num < 2:
            return False
        for i in range(2, int(math.sqrt(num)) + 1):
            if num % i == 0:
                return False
        return True

    prime = random.randint(2, 100)
    while not is_prime(prime):
        prime = random.randint(2, 100)
    return prime

def generate_points(n, p):
    points = set()
    while len(points) < n:
        x = random.randint(0, p - 1)
        y = random.randint(0, p - 1)
        if (x, y) not in points:
            points.add((x, y))
    return list(points)

def generate_circuit(points):
    # Simplified AC^0 circuit generation for demonstration
    circuit = []
    for point in points:
        x, y = point
        circuit.append(f"AND({x}, {y})")
    return circuit

def p_adic_valuation(point, p):
    x, y = point
    val_x = 0 if x == 0 else int(math.log2(x)) + 1
    val_y = 0 if y == 0 else int(math.log2(y)) + 1
    return min(val_x, val_y)

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [next_prime() for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_valuation = sum(result["metric_value"] for result in results) / len(results)
    std_deviation = math.sqrt(sum((result["metric_value"] - mean_valuation) ** 2 for result in results) / len(results))

```

```

support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_valuation} std={std_deviation} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_valuation} std={std_deviation} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify if the conjecture is supported or falsified according to the pre-registered criteria. | next: Retry the test with a longer timeout and ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14236
2	preregistration	ollama_remote	glm4:latest	0	0	9581
3	novelty	ollama_remote	glm4:latest	0	0	8840
4	novelty	ollama_remote	glm4:latest	0	0	9602
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11975
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10613
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10879
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9920
9	judge	ollama_remote	glm4:latest	0	0	11619

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97265 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/d6756f053a72.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d6756f053a72.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d6756f053a72.tar.gz` (if generated)